

06. Testing - Học Và Nhớ Library74 Từ Gốc

Tài liệu này xem bạn là người mới học lập trình web ở mức cơ bản. Mục tiêu không phải học thuộc tên công cụ test, mà là hiểu kiểm thử phần mềm đang giải quyết vấn đề gì, vì sao cần nhiều tầng test, test nào phù hợp với frontend, backend, database, AI, deploy, và khi hội đồng hỏi thì bạn giải thích được bằng logic.

Lộ trình học toàn project:

1. FE - Frontend: người dùng nhìn thấy gì, bấm gì, dữ liệu đi đâu.
2. DB - Database: dữ liệu được lưu như thế nào.
3. BE - Backend: nghiệp vụ, bảo mật, API.
4. AI - Recommendation, semantic search, xử lý tri thức.
5. Deploy/DevOps - đưa hệ thống chạy thật trên server.
6. Testing - chứng minh hệ thống hoạt động đúng và giảm lỗi khi thay đổi.

Trong file này ta học phần Testing.

1. Testing Là Gì?

1.1. Vấn Đề Thực Tế

Khi làm một hệ thống như Library74, code chạy được chưa đủ.

Ta còn phải trả lời:

- Đăng ký có chặn email trùng không?
- Đăng nhập có lưu token đúng không?
- User có thấy đúng ngôn ngữ Việt/English không?
- Thủ thư có mượn/trả sách đúng nghiệp vụ không?
- Phí phạt có tính sai không?
- Search có gọi đúng API không?
- AI service lỗi thì backend có fallback không?
- Database migration có làm vỡ native SQL không?
- Deploy xong web còn chạy đúng không?

Nếu chỉ bấm thử bằng tay, ta có thể bỏ sót lỗi. Khi sửa một chỗ, một chức năng cũ có thể hỏng lại mà ta không biết.

1.2. Định Nghĩa

Testing là quá trình kiểm tra phần mềm để phát hiện lỗi, xác minh hành vi đúng, và tạo sự tự tin khi thay đổi code.

Nói đơn giản:

Code có kỳ vọng -> chạy kiểm tra -> so sánh kết quả thật với kỳ vọng

Ví dụ kỳ vọng:

Nếu đăng ký email đã tồn tại, hệ thống phải báo email đã được sử dụng.

Test sẽ kiểm tra:

Input: email đã tồn tại
Output: lỗi EMAIL_ALREADY_EXISTS
UI: hiện đúng thông báo theo ngôn ngữ đang chọn

1.3. Testing Có Phải Là Một Môn Lớn Không?

Có. Testing là một mảng lớn trong kỹ nghệ phần mềm.

Nó bao gồm:

- Unit testing.
- Integration testing.
- End-to-end testing.
- Manual testing.
- Regression testing.
- Contract testing.
- Performance testing.
- Security testing.
- Usability testing.
- Test data management.
- CI/CD testing.

Nhưng với đồ án web, ta không cần học tất cả thật sâu ngay từ đầu. Ta cần hiểu các tầng test quan trọng và biết áp dụng đúng vào project.

Trả lời hội đồng:

Testing là quá trình xác minh hệ thống hoạt động đúng theo kỳ vọng và phát hiện lỗi trước khi người dùng gặp lỗi. Với Library74, testing quan trọng vì hệ thống có nhiều nghiệp vụ như đăng nhập, tìm kiếm, đặt trước, mượn trả, phí phạt, notification, mail và AI.

2. Vì Sao Không Chỉ Test Bằng Tay?

2.1. Test Bằng Tay Là Gì?

Test bằng tay là mở web lên và tự thao tác:

```
Mở trang đăng nhập.  
Nhập email.  
Nhập password.  
Bấm đăng nhập.  
Quan sát kết quả.
```

Test bằng tay rất cần, đặc biệt với UI.

Nhưng nếu chỉ test bằng tay thì có vấn đề.

2.2. Vấn Đề Của Test Bằng Tay

Test bằng tay:

- Tốn thời gian khi phải lặp lại nhiều lần.
- Dễ quên một case.
- Khó kiểm tra hết các nhánh lỗi.
- Phụ thuộc người test.
- Không tự chạy khi code thay đổi.
- Không báo lỗi tự động trong CI.

Ví dụ:

Bạn sửa text tiếng Việt ở Contact Tickets. Sau đó vô tình làm hỏng English mode. Nếu không bấm chuyển English để test, lỗi sẽ lọt qua.

2.3. Test Tự Động Giúp Gì?

Test tự động là code kiểm tra code.

Ví dụ:

```
Chạy npm test.  
Chạy mvn test.  
Chạy pytest.
```

Máy sẽ kiểm tra nhiều case lặp lại nhanh hơn con người.

2.4. Test Bằng Tay Và Test Tự Động Bổ Sung Cho Nhau

Không nên nghĩ:

```
Có test tự động thì không cần test bằng tay.
```

Thực tế:

- Test tự động tốt để bắt lỗi logic lặp lại.

- Test bằng tay tốt để cảm nhận UI, layout, flow thật.
- E2E test tốt để kiểm tra luồng tổng thể.

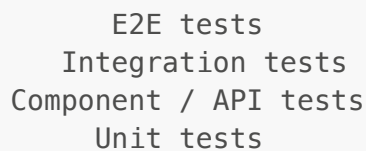
Trả lời hội đồng:

Test thủ công giúp kiểm tra trải nghiệm thật, nhưng dễ bỏ sót và khó lặp lại. Test tự động giúp kiểm tra nhanh các hành vi quan trọng mỗi khi sửa code. Trong project, em kết hợp cả hai: test tự động cho logic, API contract, use case, và test thủ công cho UI workflow sau deploy.

3. Test Pyramid

3.1. Khái Niệm

Test pyramid là cách tổ chức test theo nhiều tầng.



```
E2E tests
Integration tests
Component / API tests
Unit tests
```

Ý tưởng:

- Unit test nhiều nhất vì chạy nhanh.
- Integration test ít hơn vì nặng hơn.
- E2E test ít nhất vì chậm và dễ phụ thuộc môi trường.

3.2. Unit Test

Unit test kiểm tra một hàm, một class, một use case nhỏ.

Ví dụ trong frontend:

```
reservationUtils.test.ts
errorMessages.test.ts
notificationLocalization.test.ts
```

Ví dụ trong backend:

```
BorrowRequestUseCaseTest.java
ReturnBookUseCaseTest.java
CreateReservationUseCaseTest.java
```

Ví dụ trong AI:

```
test_recommender_engine.py
test_pipeline_core.py
test_config.py
```

3.3. Component/API Test

Component test kiểm tra một phần UI hoặc context.

Ví dụ:

```
LanguageContext.test.tsx
ThemeContext.test.tsx
AuthContext.test.tsx
UiComponents.test.tsx
```

API/controller test kiểm tra controller backend bằng MockMvc.

Ví dụ:

```
AuthControllerTest.java
PublicationControllerTest.java
FineControllerTest.java
```

3.4. Integration Test

Integration test kiểm tra nhiều phần kết hợp.

Ví dụ:

```
Backend + PostgreSQL thật bằng Testcontainers
Migration Flyway + native SQL
```

Trong project có:

```
ReservationNativeSqlIntegrationTest.java
BorrowFlowIntegrationTest.java
```

3.5. End-to-End Test

E2E test kiểm tra từ góc nhìn người dùng:

Mở browser -> đăng nhập -> search sách -> đặt trước -> xem notification

Công cụ thường dùng:

Playwright
Cypress
Selenium

Project hiện ưu tiên unit/component/integration. Playwright E2E là hướng nên bổ sung cho các luồng chính.

Trả lời hội đồng:

Em áp dụng tư duy test pyramid. Unit test dùng để kiểm tra logic nhỏ và chạy nhanh, integration test kiểm tra các phần kết nối như database thật, còn E2E test kiểm tra luồng người dùng thật. Cách này cân bằng giữa tốc độ, độ tin cậy và phạm vi kiểm tra.

4. Testing Trong Library74 Đang Bao Phủ Gì?

Project có các tài liệu test:

TEST_PLAN.md
TEST_REPORT.md
TEST_CASES.md
LMS_BE/TEST_REPORT.md
LMS_LATEX/tex/chapters/07_testing.tex

Project có test ở ba phần chính:

Phần	Công cụ	Mục tiêu
Frontend	Jest, Testing Library, jsdom	Test helper, context, component, service contract
Backend	JUnit 5, Mockito, MockMvc, Spring Boot Test	Test use case, controller, API, integration
AI	pytest, FastAPI TestClient/httpx	Test API contract, pipeline, recommender, fallback

Các lệnh chính:

```
cd LMS_FE  
npm test -- --runInBand
```

```
cd LMS_BE  
mvn test
```

```
cd LMS_AI  
pytest
```

Trả lời hội đồng:

Library74 có test cho frontend, backend và AI. Frontend dùng Jest/Testing Library để test logic giao diện và context. Backend dùng JUnit/Mockito/MockMvc để test nghiệp vụ và API. AI dùng pytest để test contract, pipeline và recommendation. Ngoài ra còn có test plan, test report và chương testing trong báo cáo.

5. Frontend Testing

5.1. Frontend Cần Test Gì?

Frontend không chỉ là giao diện đẹp.

Frontend còn có logic:

- Map lỗi backend sang thông báo người dùng.
- Chuyển tiếng Việt/English.
- Lưu theme light/dark/system.
- Lưu token đăng nhập.
- Gọi đúng endpoint backend.
- Format ngày giờ, trạng thái đặt trước.
- Hiển thị notification đúng ngôn ngữ.
- Component button/input/modal hoạt động đúng.

5.2. Công Cụ Frontend

Trong `LMS_FE/package.json` có:

```
"test": "jest --config jest.config.cjs",  
"test:watch": "jest --config jest.config.cjs --watch",  
"test:coverage": "jest --config jest.config.cjs --coverage"
```

Các công cụ chính:

Công cụ	Vai trò
Jest	Test runner, chạy test và assertion
Testing Library	Test component theo hành vi người dùng
jsdom	Giả lập DOM trong Node.js
ts-jest	Biên dịch TypeScript khi test
jest-dom	Assertion cho DOM như <code>toHaveTextContent</code>

5.3. jsdom Là Gì?

Test frontend chạy trong Node.js, không phải browser thật.

Những component React cần:

- `document`
- `window`
- `localStorage`
- DOM element

`jsdom` giả lập môi trường trình duyệt để test có thể chạy.

5.4. Testing Library Tư Duy Như Thế Nào?

Testing Library khuyến khích test theo cách người dùng nhìn và thao tác.

Không nên chỉ test implementation detail.

Ví dụ tốt:

```
Tìm nút theo role button.
Bấm vào nút.
Kỳ vọng text thay đổi.
```

Ví dụ kém:

```
Kiểm tra state nội bộ của component.
```

5.5. Các Test Frontend Trong Project

Một số file quan trọng:

```
LMS_FE/src/utils/errorMessages.test.ts
LMS_FE/src/utils/notificationLocalization.test.ts
LMS_FE/src/utils/reservationUtils.test.ts
LMS_FE/src/contexts/LanguageContext.test.tsx
```

```
LMS_FE/src/contexts/ThemeContext.test.tsx
LMS_FE/src/contexts/AuthContext.test.tsx
LMS_FE/src/contexts/UploadContext.test.tsx
LMS_FE/src/components/UiComponents.test.tsx
LMS_FE/src/components/Switchers.test.tsx
LMS_FE/src/pages/ContactPage.test.tsx
LMS_FE/src/api/frontendServices.test.ts
```

5.6. Ví Dụ Logic Lỗi Ngôn Ngữ

Một lỗi từng gặp:

Duplicate email nhưng frontend lại hiện thông báo duplicate review.

Nguyên nhân:

Frontend map chuỗi "already exists" quá chung.

Cách test đúng:

Input: backend trả code EMAIL_ALREADY_EXISTS
Kỳ vọng English: This email is already registered.
Kỳ vọng Vietnamese: Email đã được sử dụng.

Ý nghĩa:

- Test giúp lỗi này không tái xuất hiện.
- Khi backend đổi message text nhưng giữ code, frontend vẫn đúng.

5.7. Ví Dụ Test Context

LanguageContext cần đảm bảo:

- Default language đúng.
- Chuyển sang English được.
- Lưu vào **localStorage**.
- Cập nhật **document.lang**.
- Nếu thiếu key dịch thì fallback hợp lý.

ThemeContext cần đảm bảo:

- Light mode đúng.
- Dark mode đúng.
- System mode theo **prefers-color-scheme**.
- Class **dark** trên thẻ **html** được cập nhật đúng.

5.8. Service Contract Test Ở Frontend

Frontend gọi API qua service layer.

Test service contract kiểm tra:

- URL đúng không.
- Method đúng không.
- Query string đúng không.
- Payload đúng không.

Ví dụ:

```
Search sách phải gọi đúng endpoint search.  
Wishlist phải gọi đúng endpoint wishlist.  
Rating phải gửi đúng body.  
Admin audit phải gọi đúng route.
```

Service contract test không cần backend thật. Nó mock axios/fetch và kiểm tra frontend chuẩn bị request đúng.

Trả lời hội đồng:

Frontend testing của project tập trung vào logic có ảnh hưởng rộng: đa ngôn ngữ, theme, auth context, notification localization, error mapping và service contract. Các test này giúp giảm lỗi khi sửa UI vì nhiều trang dùng chung các context và helper đó.

6. Backend Testing

6.1. Backend Cần Test Gì?

Backend chứa nghiệp vụ quan trọng nhất:

- Đăng ký, đăng nhập.
- Phân quyền user/librarian/admin.
- Tạo sách, bản sao sách.
- Tìm kiếm publication.
- Đặt trước sách.
- Mượn sách.
- Trả sách.
- Tính phí phạt.
- Wishlist/rating.
- Notification.
- AI callback.
- Gửi mail.

Nếu backend sai, UI đẹp vẫn không cứu được nghiệp vụ.

6.2. Công Cụ Backend

Project dùng:

Công cụ	Vai trò
JUnit 5	Framework chạy test Java
Mockito	Mock dependency
MockMvc	Test controller HTTP mà không cần chạy server thật
Spring Boot Test	Test trong Spring context
Testcontainers	Chạy PostgreSQL thật trong Docker để integration test
Maven Surefire	Plugin chạy test khi <code>mvn test</code>

6.3. Unit Test Use Case

Use case là nơi chứa nghiệp vụ.

Ví dụ:

```
BorrowRequestUseCaseTest.java
ReturnBookUseCaseTest.java
CreateReservationUseCaseTest.java
CreatePublicationRatingUseCaseTest.java
ClearWishlistUseCaseTest.java
GetRecommendationsUseCaseTest.java
```

Các test này thường mock repository/service phụ.

Ví dụ logic:

```
Nếu user đã mượn quá giới hạn, borrow request phải bị chặn.
Nếu sách không có item available, reservation được tạo.
Nếu user chưa từng mượn publication, không được rating.
Nếu AI lỗi, recommendation fallback về trending.
```

6.4. Mockito Là Gì?

Mockito dùng để tạo object giả.

Ví dụ use case cần repository:

```
UseCase -> Repository -> Database
```

Unit test không muốn gọi database thật. Ta mock repository:

```
Khi repository.findByEmail(email) được gọi, trả về user có sẵn.
```

Sau đó kiểm tra use case xử lý đúng.

6.5. Controller Test Bằng MockMvc

Controller là cửa vào API.

Ví dụ:

```
POST /api/auth/register
GET /api/publications
POST /api/ratings
GET /api/fines
```

MockMvc giúp test:

- HTTP status.
- Request body validation.
- Response body.
- Controller gọi đúng use case.
- Security/role ở một mức nhất định.

Ví dụ:

```
AuthControllerTest.java
NotificationControllerTest.java
PublicationControllerTest.java
BorrowingTransactionControllerTest.java
FineControllerTest.java
```

6.6. Integration Test Với Database Thật

Một số lỗi chỉ xuất hiện khi có database thật:

- Native SQL sai tên cột.
- Migration thiếu bảng.
- Query chạy trên H2 được nhưng PostgreSQL không chạy.
- Constraint database khác expectation.
- Flyway migration vỡ.

Vì vậy project có integration test với Testcontainers.

Ví dụ:

```
ReservationNativeSqlIntegrationTest.java  
BorrowFlowIntegrationTest.java
```

Testcontainers sẽ chạy PostgreSQL trong Docker, sau đó test chạy trên schema thật.

6.7. Vì Sao Không Chỉ Dùng H2?

H2 là database nhẹ, thường dùng cho test.

Nhưng production dùng PostgreSQL. PostgreSQL có:

- Kiểu dữ liệu khác.
- SQL dialect khác.
- Extension như pgvector.
- Behavior constraint khác.

Nếu project có native SQL, dùng PostgreSQL thật trong test an toàn hơn.

Trả lời hội đồng:

Backend testing của project chia thành unit test cho use case, controller test bằng MockMvc và integration test với PostgreSQL thật bằng Testcontainers. Unit test chạy nhanh để kiểm nghiệm vụ, controller test kiểm hợp đồng API, còn integration test bắt lỗi migration và native SQL mà compile không phát hiện được.

7. AI Testing

7.1. Vì Sao AI Cần Test Riêng?

AI khác backend thường ở chỗ kết quả có thể không tuyệt đối cố định.

Ví dụ recommendation:

```
User A có thể nhận danh sách sách khác tùy dữ liệu interaction.
```

Semantic search:

```
Query "machine learning" phải trả sách liên quan, nhưng thứ tự có thể phụ  
thuộc embedding/model/data.
```

Vì vậy AI testing cần kiểm tra cả:

- Contract input/output.
- Pipeline có chạy không.
- Fallback có hoạt động không.

- Latency có vượt ngưỡng không.
- Chất lượng kết quả có chấp nhận được không.

7.2. Công Cụ AI

Project dùng:

Công cụ	Vai trò
pytest	Test runner Python
FastAPI TestClient/httpx	Test API
Mock/stub	Giả lập dependency nặng
Dataset mẫu	Đánh giá kết quả AI ổn định hơn

Các test trong project:

```
LMS_AI/tests/test_api_service_contract.py
LMS_AI/tests/test_worker_contract.py
LMS_AI/tests/test_pdf_processing.py
LMS_AI/tests/test_config.py
LMS_AI/tests/test_ai_metadata_quality.py
LMS_AI/tests/test_database_sql_contract.py
LMS_AI/tests/test_llm_client.py
LMS_AI/tests/test_recommender_engine.py
LMS_AI/tests/test_pipeline_core.py
LMS_AI/tests/integration/test_api_contract_and_latency.py
```

7.3. AI Contract Test

AI contract test kiểm tra API trả đúng schema.

Ví dụ:

```
/health phải trả {"status": "ok"}
/api/v1/semantic-search phải trả publication_ids
/api/v1/recommendations phải trả publication_ids và strategy
```

Mục tiêu:

- Backend gọi AI không bị vỡ vì đổi tên field.
- AI service có endpoint đúng.
- Response format ổn định.

7.4. Fallback Test

AI có thể lỗi vì:

- Model chưa load.
- Vector chưa có dữ liệu.
- Gateway timeout.
- API key hết quota.
- Worker đang bận.

Core library workflow không nên chết hoàn toàn vì AI lỗi.

Vì vậy cần test:

Nếu AI unavailable, backend fallback về trending hoặc cached recommendation.
Nếu semantic search không có vector, hệ thống vẫn trả response hợp lệ.

7.5. AI Quality Test

AI không chỉ cần "không crash".

Nó còn cần kết quả có ý nghĩa.

Một cách đơn giản:

Tạo dataset mẫu.
Với query A, kỳ vọng top-k có sách liên quan.
Đo precision@k hoặc kiểm thủ công có bằng chứng.

Ví dụ:

Query: data structure
Expected relevant books: Data Structures and Algorithms, Introduction to Algorithms

Nếu top 5 có các sách đó, semantic search được xem là hợp lý.

Trả lời hội đồng:

AI testing không chỉ kiểm tra endpoint chạy được, mà còn kiểm tra contract, fallback, latency và chất lượng kết quả. Với Library74, điều quan trọng là AI lỗi không làm hỏng nghiệp vụ thư viện chính, và kết quả semantic search/recommendation phải có bằng chứng đánh giá trên dữ liệu mẫu.

8. Database Testing

8.1. Database Có Gì Cần Test?

Database không chỉ lưu dữ liệu.

Database còn có:

- Table.
- Column.
- Foreign key.
- Unique constraint.
- Index.
- Migration.
- Native SQL.
- Seed data.
- pgvector.

Nếu database sai, backend có thể compile vẫn pass nhưng runtime fail.

8.2. Migration Test

Migration là script thay đổi schema.

Ví dụ:

```
V1__init.sql  
V20__...  
V22__...
```

Migration test đảm bảo:

- Chạy migration từ đầu không lỗi.
- Bảng/cột cần thiết tồn tại.
- Constraint đúng.
- Native SQL dùng đúng schema.

8.3. Native SQL Test

Native SQL nguy hiểm hơn query type-safe vì Java compile không bắt được lỗi SQL.

Ví dụ lỗi:

```
Đổi tên column trong migration.  
Native SQL vẫn gọi tên column cũ.  
Compile pass.  
Runtime fail.
```

Integration test với PostgreSQL thật giúp bắt lỗi này.

8.4. Test Data

Test cần dữ liệu mẫu.

Ví dụ:

```
User A.
Publication B.
Item C.
Reservation D.
Borrowing transaction E.
Fine F.
```

Test data tốt phải:

- Đủ nhỏ để dễ hiểu.
- Đủ thực tế để bắt lỗi.
- Có thể tạo lại nhiều lần.
- Không phụ thuộc dữ liệu production.

Trả lời hội đồng:

Database testing giúp phát hiện lỗi schema, migration và native SQL. Vì production dùng PostgreSQL, integration test nên chạy với PostgreSQL thật qua Testcontainers thay vì chỉ dựa vào database giả. Điều này đặc biệt quan trọng khi hệ thống có nhiều nghiệp vụ mượn trả và truy vấn phức tạp.

9. Contract Testing

9.1. Contract Là Gì?

Contract là "hợp đồng" giữa hai phần hệ thống.

Ví dụ:

```
Frontend gọi backend.
Backend trả JSON.
Frontend kỳ vọng field tên là "publicationId".
```

Nếu backend đổi field thành **"bookId"** nhưng frontend chưa sửa, UI sẽ lỗi.

9.2. Contract Trong Library74

Các contract quan trọng:

Contract	Ví dụ
FE - BE	Frontend gọi đúng endpoint, backend trả đúng response
BE - AI	Backend gọi AI recommendation/semantic search đúng schema
BE - DB	Repository/native SQL khớp schema database

Contract	Ví dụ
BE - SMTP	Backend gửi mail với config Zoho đúng
Caddy - BE/FE	<code>/api/*</code> route vào backend, còn lại vào frontend

9.3. Service Contract Test Ở FE

Frontend service test kiểm tra frontend chuẩn bị request đúng:

```
URL đúng.  
Method đúng.  
Payload đúng.  
Query param đúng.
```

9.4. Gateway Contract Test Ở BE

Backend có thể test AI gateway bằng local HTTP stub.

Mục tiêu:

```
Nếu AI trả response đúng schema, backend parse đúng.  
Nếu AI unavailable, backend fallback đúng.
```

9.5. API Contract Test Ở AI

AI test đảm bảo endpoint trả shape mà backend đang cần.

Ví dụ:

```
recommendations response phải có publication_ids và strategy.
```

Trả lời hội đồng:

Contract testing kiểm tra ranh giới giữa các phần của hệ thống. Với Library74, nó quan trọng vì frontend, backend, AI, database và mail đều giao tiếp qua hợp đồng dữ liệu. Nếu một bên đổi schema hoặc endpoint, contract test giúp phát hiện sớm trước khi lỗi lan ra production.

10. Regression Testing

10.1. Regression Là Gì?

Regression là lỗi cũ quay lại hoặc chức năng cũ bị hỏng sau khi sửa code mới.

Ví dụ:

Sửa Contact Tickets.
Nhưng làm hỏng LanguageSwitcher.

Hoặc:

Sửa rating duplicate.
Nhưng duplicate email lại hiện message sai.

10.2. Vì Sao Regression Dễ Xảy Ra?

Vì nhiều phần dùng chung code:

- `LanguageContext`.
- `ThemeContext`.
- `AuthContext`.
- API service layer.
- Error mapping.
- Notification localization.
- Shared backend module.
- Database schema.

Sửa một helper có thể ảnh hưởng nhiều trang.

10.3. Regression Test Là Gì?

Regression test là test được giữ lại để đảm bảo lỗi đã sửa không quay lại.

Ví dụ:

Sau khi fix duplicate email message, thêm test cho duplicate email.

Sau này mỗi lần chạy test, case đó được kiểm lại.

Trả lời hội đồng:

Regression testing giúp đảm bảo chức năng cũ không bị hỏng khi sửa chức năng mới. Trong project, các test về error message, language, theme, notification và service contract đều đóng vai trò regression test cho những phần dùng chung nhiều nơi.

11. Mock, Stub, Fake Và Test Double

11.1. Vì Sao Cần Giả Lập?

Khi test một phần nhỏ, ta không muốn gọi mọi dependency thật.

Ví dụ test đăng ký user:

```
Không cần gửi mail thật.  
Không cần gọi database thật.  
Không cần publish Kafka thật.
```

Ta chỉ cần kiểm tra use case xử lý đúng.

11.2. Mock Là Gì?

Mock là object giả dùng để kiểm tra cách code tương tác với dependency.

Ví dụ:

```
Kiểm tra emailPublisher đã được gọi sau khi đăng ký thành công.
```

11.3. Stub Là Gì?

Stub là object giả trả dữ liệu cố định.

Ví dụ:

```
Khi userRepository.findByEmail(email), trả về Optional.empty().
```

11.4. Fake Là Gì?

Fake là implementation đơn giản dùng cho test.

Ví dụ:

```
In-memory repository lưu dữ liệu trong list.
```

11.5. Khi Nào Không Nên Mock?

Không nên mock mọi thứ.

Nếu mục tiêu là kiểm tra database thật, phải dùng database thật.

Nếu mục tiêu là kiểm tra route frontend thật trên browser, phải dùng browser thật.

Mock phù hợp với unit test, không thay thế integration test.

Trả lời hội đồng:

Mock, stub và fake giúp cô lập phần đang test để test chạy nhanh và ổn định. Tuy nhiên không nên mock tất cả, vì những lỗi ở database, network hoặc tích hợp service chỉ được phát hiện bằng integration hoặc E2E test.

12. Coverage Là Gì?

12.1. Định Nghĩa

Coverage là tỷ lệ code được test chạy qua.

Ví dụ:

80% line coverage nghĩa là 80% dòng code được chạy khi test.

Frontend có thể chạy:

```
cd LMS_FE  
npm run test:coverage
```

Backend có thể cấu hình JaCoCo nếu muốn đo coverage Java.

12.2. Coverage Cao Có Đảm Bảo Không Có Lỗi Không?

Không.

Coverage chỉ nói code đã được chạy qua, không nói assertion có đúng hay không.

Ví dụ test kém:

Render component nhưng không assert gì.

Coverage có thể tăng, nhưng không chứng minh hành vi đúng.

12.3. Coverage Nên Dùng Như Thế Nào?

Coverage nên dùng để phát hiện vùng chưa được test.

Nhưng chất lượng test quan trọng hơn số phần trăm.

Nên ưu tiên:

- Nghiệp vụ quan trọng.
- Logic dễ lỗi.
- Code dùng chung.
- Luồng tiền/phí/phạt.
- Auth/security.

- AI fallback.
- Database native SQL.

Trả lời hội đồng:

Coverage là chỉ số tham khảo để biết vùng code nào đã được test chạy qua, nhưng không thay thế chất lượng test. Một test tốt phải có input rõ, kỳ vọng rõ và assertion bắt được lỗi nghiệp vụ thật.

13. E2E Testing

13.1. E2E Là Gì?

E2E là kiểm thử từ đầu đến cuối như người dùng thật.

Ví dụ:

```
Mở browser.  
Đăng nhập.  
Tìm sách.  
Đặt trước.  
Xem notification.  
Logout.
```

E2E kiểm tra:

- Frontend render đúng.
- Backend API chạy đúng.
- Auth token hoạt động.
- Database có dữ liệu.
- Browser thật tương tác được.

13.2. Công Cụ

Công cụ phổ biến:

```
Playwright  
Cypress  
Selenium
```

Với project React hiện đại, Playwright là lựa chọn tốt.

13.3. E2E Nên Test Ít Nhưng Chất

Không nên viết E2E cho mọi chi tiết nhỏ.

Nên viết cho critical path:

- Register/login.

- Search sách.
- Reserve sách.
- Librarian xử lý mượn/trả.
- User xem phí phạt.
- Contact ticket.
- Admin quản lý tài khoản.
- Đổi language/theme.

13.4. Vì Sao E2E Dễ Flaky?

E2E phụ thuộc nhiều thứ:

- Browser.
- Network.
- Backend.
- Database seed.
- Animation/loading.
- Timing.

Vì vậy E2E cần:

- Test data ổn định.
- Selector rõ.
- Chờ theo trạng thái thật, không chờ thời gian cứng.
- Reset database hoặc seed data trước test.

Trả lời hội đồng:

E2E test mô phỏng người dùng thật trên browser và kiểm tra cả hệ thống từ frontend đến backend và database. Tuy nhiên E2E chậm và dễ flaky, nên chỉ nên dùng cho các luồng nghiệp vụ quan trọng thay vì thay thế unit test.

14. Manual Testing Và Checklist

14.1. Khi Nào Cần Manual Test?

Manual test vẫn rất quan trọng khi kiểm tra:

- UI có đẹp và dễ dùng không.
- Responsive mobile/desktop.
- Dark mode có bị lệch màu không.
- Text có tràn không.
- Modal có bị che không.
- Flow có hợp lý không.
- Mail có vào inbox thật không.
- Production sau deploy có hoạt động không.

14.2. Checklist Manual Cho Library74

Sau deploy hoặc trước demo, nên kiểm:

- Trang chủ mở được.
- Đăng ký/đăng nhập.
- Đổi tiếng Việt/English.
- Đổi light/dark/system.
- User search sách.
- User xem dạng grid/list nếu trang hỗ trợ.
- User tạo contact ticket.
- Librarian xử lý contact inbox.
- Librarian mượn/trả sách.
- User nhận notification.
- Mail **noreply** cho thao tác hệ thống.
- Mail **support** cho hoàn tất hỗ trợ.
- Admin vào trang quản lý tài khoản.
- AI semantic search/recommendation.

14.3. Manual Test Không Nên Chỉ Ghi "Pass"

Nên ghi:

Ngày test.
Môi trường test.
Tài khoản test.
Các bước test.
Kết quả kỳ vọng.
Kết quả thực tế.
Screenshot nếu cần.
Lỗi còn lại.

Trả lời hội đồng:

Manual testing dùng để kiểm tra trải nghiệm thật mà unit test khó thấy, như layout, responsive, dark mode, flow thao tác và mail thật. Tuy nhiên manual test nên có checklist và bằng chứng, không chỉ ghi chung chung là đã test.

15. Performance Testing

15.1. Performance Là Gì?

Performance testing kiểm tra hệ thống có nhanh và chịu tải được không.

Các chỉ số:

- Response time.
- Throughput.
- Error rate.
- Memory usage.

- CPU usage.
- Database query time.
- AI latency.

15.2. Với Library74 Cần Quan Tâm Gì?

Các điểm dễ chậm:

- Search sách.
- Dashboard thống kê.
- Native SQL.
- Semantic search.
- Recommendation.
- PDF processing.
- Gửi mail hàng loạt.
- WebSocket notification.

15.3. Performance Test Đơn Giản

Có thể bắt đầu bằng:

```
curl -w "%{time_total}\n" -o /dev/null -s https://library74.uk/api/...
```

Hoặc dùng tool:

```
k6  
JMeter  
Gatling  
Locust
```

15.4. AI Latency

AI latency cần đo riêng vì model và embedding có thể chậm.

Ví dụ:

```
semantic search dưới 2 giây  
recommendation dưới 1 giây  
PDF ETL có thể chạy nền, không cần trả ngay
```

Trả lời hội đồng:

Performance testing kiểm tra tốc độ và khả năng chịu tải. Với Library74, các điểm cần chú ý là search, dashboard, semantic search, recommendation và PDF processing. Những tác vụ AI nặng nên chạy nền qua worker để không làm nghẽn request người dùng.

16. Security Testing

16.1. Security Testing Là Gì?

Security testing kiểm tra hệ thống có lỗ hổng bảo mật không.

Với web app, cần quan tâm:

- Auth.
- Role-based access control.
- JWT.
- XSS.
- CSRF.
- SQL injection.
- Secret leak.
- Upload file.
- API public/private.
- Callback verification.

16.2. Security Case Trong Library74

Các case quan trọng:

- User không được truy cập trang librarian/admin.
- Librarian không được làm quyền admin.
- API admin phải yêu cầu role admin.
- JWT hết hạn phải bị từ chối.
- Password không lưu plain text.
- OAuth secret không hardcode.
- `.env.prod` không commit.
- `/api/ai/callback` phải xác minh HMAC.
- Upload PDF phải kiểm soát file type/size.

16.3. Test Phân Quyền

Ví dụ:

```
User gọi API admin -> phải nhận 403.  
Không có token -> phải nhận 401.  
Token hợp lệ nhưng sai role -> phải nhận 403.
```

16.4. Test Secret

Không nên để test in ra:

```
MAIL_SENDER_PASSWORD  
JWT_SECRET
```

```
GOOGLE_CLIENT_SECRET
API key
```

Log test phải redact secret nếu cần.

Trả lời hội đồng:

Security testing đảm bảo người dùng chỉ làm được đúng quyền của mình và secret không bị lộ. Với Library74, các test quan trọng là auth, phân quyền user/librarian/admin, JWT, callback HMAC, upload file và không commit `.env.prod`.

17. Test Data Và Test Environment

17.1. Test Environment Là Gì?

Test environment là môi trường dùng để chạy test.

Ví dụ:

- Local.
- Docker test.
- Staging.
- Production smoke test.

17.2. Không Dùng Production Data Cho Test Tự Do

Không nên test phá dữ liệu trên production.

Ví dụ không nên:

```
Tạo lung tung user thật.
Xóa sách thật.
Tạo phí phạt giả.
```

Nên có:

- Tài khoản test.
- Dữ liệu seed.
- Database test.
- Script reset data nếu cần.

17.3. Test Data Tốt

Test data nên có:

- User thường.
- Librarian.
- Admin.

- Publication có item available.
- Publication không có item available.
- User đang mượn sách.
- User có phí phạt.
- Reservation đang pending.
- Contact ticket open/resolved.
- Dữ liệu interaction cho recommendation.

17.4. Test Isolation

Test tốt không phụ thuộc thứ tự chạy.

Không nên:

Test B chỉ pass nếu Test A chạy trước.

Nên:

Mỗi test tự tạo dữ liệu cần thiết hoặc reset trạng thái.

Trả lời hội đồng:

Test data phải được quản lý riêng để test có thể lặp lại và không phá production. Một test tốt phải độc lập, có dữ liệu đầu vào rõ ràng và không phụ thuộc thứ tự chạy của test khác.

18. Smoke Test Sau Deploy

18.1. Smoke Test Là Gì?

Smoke test là kiểm tra nhanh sau deploy để xem hệ thống có "sống" không.

Nó không thay thế toàn bộ test suite.

Nó trả lời:

Deploy xong web có mở được không?
API có sống không?
Backend health UP không?
AI health ok không?
Chức năng chính có chạy không?

18.2. Smoke Test Library

Sau deploy:

```
curl -I https://library74.uk
```

Backend:

```
docker exec lms_backend wget -qO- http://localhost:8080/actuator/health
```

AI:

```
docker exec lms_ai_api curl -fsS http://localhost:8001/health
```

Browser:

Mở trang chủ.
Login.
Search.
Kiểm tra trang vừa sửa.

18.3. Khi Smoke Test Fail

Nếu website 502:

Xem log caddy.
Xem log backend.
Kiểm tra backend health.

Nếu mail fail:

Xem log backend.
Kiểm tra MAIL_* trong .env.prod.
Kiểm tra app password Zoho.

Nếu frontend không đổi:

Build lại frontend.
Hard refresh browser.
Kiểm tra container frontend đã recreate chưa.

Trả lời hội đồng:

Smoke test là bước kiểm nhanh sau deploy để đảm bảo hệ thống vẫn sống. Với Library74, em kiểm tra website, backend health, AI health và chức năng vừa sửa. Nếu fail thì đọc log đúng service để tìm nguyên nhân.

19. CI/CD Và Testing

19.1. CI Là Gì?

CI là Continuous Integration.

Ý tưởng:

Mỗi khi push code, hệ thống tự chạy build và test.

Nếu test fail, không nên merge/deploy.

19.2. CD Là Gì?

CD là Continuous Delivery hoặc Continuous Deployment.

Ý tưởng:

Code đã pass test có thể được đưa lên môi trường deploy.

19.3. Pipeline Cơ Bản Cho Library74

Một pipeline hợp lý:

```
Install FE dependencies
Run FE tests
Build FE
Run BE tests
Compile BE
Run AI tests
Build Docker images
Deploy staging/production
Smoke test
```

19.4. Vì Sao CI Quan Trọng?

CI giúp:

- Không quên chạy test.
- Phát hiện lỗi sớm.
- Giữ chất lượng khi nhiều người cùng sửa.

- Có bằng chứng test trong lịch sử commit.

Trả lời hội đồng:

CI tự động chạy build và test khi code thay đổi. Điều này giúp phát hiện lỗi trước khi merge hoặc deploy. Với Library74, pipeline nên chạy test FE, BE, AI, build frontend/backend và sau deploy chạy smoke test.

20. Cách Viết Một Test Tốt

20.1. Cấu Trúc Arrange - Act - Assert

Một test dễ hiểu thường có ba phần:

Arrange: chuẩn bị dữ liệu.
Act: gọi hàm hoặc thao tác cần test.
Assert: kiểm tra kết quả.

Ví dụ logic:

Arrange: user đã có email tồn tại.
Act: gọi đăng ký với email đó.
Assert: nhận lỗi EMAIL_ALREADY_EXISTS.

20.2. Tên Test Phải Nói Được Hành Vi

Tên test nên mô tả:

Khi điều kiện nào, hệ thống phải làm gì.

Ví dụ tốt:

```
shouldRejectDuplicateEmail  
shouldFallbackToTrendingWhenAiGatewayUnavailable  
shouldLocalizeBorrowNotificationInEnglish
```

20.3. Một Test Chỉ Nên Kiểm Một Ý Chính

Nếu một test kiểm quá nhiều thứ, khi fail sẽ khó biết lỗi ở đâu.

20.4. Test Phải Ổn Định

Tránh:

- Phụ thuộc giờ hiện tại nếu không kiểm soát clock.
- Phụ thuộc network thật trong unit test.
- Phụ thuộc thứ tự test.
- Dữ liệu random không seed.
- Sleep cứng trong E2E.

20.5. Test Phải Có Assertion Có Nghĩa

Không nên chỉ gọi hàm rồi không kiểm gì.

Test tốt phải fail khi behavior sai.

Trả lời hội đồng:

Một test tốt có dữ liệu chuẩn bị rõ, hành động rõ và kỳ vọng rõ. Em thường nghĩ theo Arrange - Act - Assert. Test phải độc lập, ổn định và có assertion bắt được lỗi nghiệp vụ thật.

21. Khi Sửa Code Thì Chạy Test Nào?

21.1. Sửa Frontend

Ví dụ sửa:

```
Component.  
Context.  
Page.  
Service API.  
i18n.
```

Chạy:

```
cd LMS_FE  
npm test -- --runInBand  
npm run build
```

Nếu sửa UI lớn, test bằng browser.

21.2. Sửa Backend

Ví dụ sửa:

```
Use case.  
Controller.  
Security.  
Mail.
```

```
Circulation.  
Recommendation gateway.
```

Chạy:

```
cd LMS_BE  
mvn test
```

Nếu muốn chạy module cụ thể:

```
mvn -pl library-circulation-module test
```

Nếu sửa migration/native SQL:

```
mvn -pl library-bootstrap -Dtest=ReservationNativeSqlIntegrationTest test
```

21.3. Sửa AI

Chạy:

```
cd LMS_AI  
pytest
```

Nếu muốn live contract test:

```
RUN_LIVE_AI_CONTRACT_TESTS=1 API_BASE_URL=http://127.0.0.1:8001 pytest  
tests/integration/test_api_contract_and_latency.py -q
```

21.4. Sửa Deploy/Env

Chạy:

```
docker compose --env-file .env.prod -f docker-compose.prod.yml config
```

Sau deploy:

```
curl -I https://library74.uk  
docker exec lms_backend wget -qO- http://localhost:8080/actuator/health
```

```
docker exec lms_ai_api curl -fsS http://localhost:8001/health
```

Trả lời hội đồng:

Khi sửa phần nào thì ưu tiên chạy test liên quan phần đó, sau đó chạy build hoặc smoke test nếu thay đổi ảnh hưởng production. Sửa frontend thì chạy Jest và build, sửa backend thì chạy Maven test, sửa AI thì chạy pytest, sửa deploy/env thì kiểm tra compose config và healthcheck sau deploy.

22. Câu Hỏi Hội Đồng Hay Hỏi

Câu 1: Vì sao cần testing?

Vì testing giúp phát hiện lỗi sớm, xác minh hệ thống hoạt động đúng theo yêu cầu và giảm rủi ro khi sửa code. Với hệ thống thư viện, lỗi mượn/trả/phí phạt hoặc phân quyền có thể ảnh hưởng dữ liệu thật nên cần test.

Câu 2: Unit test khác integration test thế nào?

Unit test kiểm một đơn vị nhỏ và thường mock dependency nên chạy nhanh. Integration test kiểm nhiều phần kết hợp, ví dụ backend với database PostgreSQL thật, nên bắt được lỗi kết nối, migration và SQL.

Câu 3: Vì sao frontend cũng cần test?

Frontend có logic thật như i18n, theme, auth, error mapping, notification localization và API service. Nếu frontend map sai lỗi hoặc gọi sai endpoint, người dùng vẫn gặp lỗi dù backend đúng.

Câu 4: Vì sao dùng mock?

Mock giúp cô lập phần đang test và tránh phụ thuộc vào database, network, mail hoặc service ngoài. Nhờ đó unit test chạy nhanh và ổn định.

Câu 5: Vì sao vẫn cần integration test nếu đã có unit test?

Unit test có thể pass vì dependency bị mock, nhưng khi kết nối thật có thể lỗi do SQL, schema, config hoặc network. Integration test kiểm tra các ranh giới thật đó.

Câu 6: Vì sao AI testing khó hơn backend thường?

Vì kết quả AI có thể phụ thuộc model, dữ liệu và embedding, không phải lúc nào cũng cố định tuyệt đối. Do đó cần kiểm contract, fallback, latency và đánh giá chất lượng bằng dataset mẫu.

Câu 7: Coverage cao có chứng minh hệ thống không lỗi không?

Không. Coverage chỉ cho biết code được chạy qua. Quan trọng hơn là test có assertion đúng và kiểm tra hành vi nghiệp vụ quan trọng.

Câu 8: Testcontainers dùng để làm gì?

Testcontainers giúp chạy database thật trong Docker khi test. Với project dùng PostgreSQL và native SQL, cách này phát hiện lỗi schema/SQL tốt hơn dùng database giả.

Câu 9: E2E test có thay thế unit test không?

Không. E2E test kiểm luồng người dùng thật nhưng chậm và dễ flaky. Unit test nhanh và phù hợp kiểm logic nhỏ. Hai loại test bổ sung cho nhau.

Câu 10: Sau deploy cần test gì?

Cần smoke test: mở website, kiểm backend health, AI health, login, search và chức năng vừa sửa. Nếu có sửa mail thì test luồng mail thật.

23. Tóm Tắt Một Phút

Testing là quá trình kiểm tra phần mềm để phát hiện lỗi và xác minh hệ thống hoạt động đúng. Với Library74, testing quan trọng vì hệ thống có nhiều nghiệp vụ: auth, tìm kiếm, đặt trước, mượn trả, phí phạt, notification, contact ticket, mail và AI.

Project áp dụng nhiều tầng test. Frontend dùng Jest, Testing Library và jsdom để test helper, context, component và service contract. Backend dùng JUnit 5, Mockito, MockMvc, Spring Boot Test và Testcontainers để test use case, controller và database integration. AI dùng pytest để test API contract, pipeline, recommender, fallback và latency. Sau deploy cần smoke test website, backend health, AI health và chức năng chính.

Điểm quan trọng là không chạy theo số lượng test đơn thuần. Test tốt phải kiểm đúng rủi ro: nghiệp vụ quan trọng, code dùng chung, contract giữa các service, database migration, security và fallback khi dependency lỗi.

24. Mười Ý Chính Cần Nhớ

1. Testing giúp phát hiện lỗi sớm và giảm rủi ro khi sửa code.
2. Manual test cần thiết nhưng không đủ vì khó lặp lại và dễ bỏ sót.
3. Test pyramid gồm unit, component/API, integration và E2E.
4. Frontend cũng cần test vì có logic i18n, theme, auth, error mapping và API service.
5. Backend cần test use case, controller, security, mail, AI gateway và database integration.
6. AI cần test contract, fallback, latency và chất lượng kết quả.
7. Mock giúp unit test nhanh, nhưng không thay thế integration test.
8. Coverage là chỉ số tham khảo, không chứng minh hệ thống không lỗi.
9. Smoke test sau deploy giúp kiểm nhanh production còn hoạt động.
10. Test tốt phải có input rõ, hành động rõ, kỳ vọng rõ và assertion có ý nghĩa.